



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3
по курсу «Языки и методы программирования»
«Полиморфизм на основе интерфейсов в языке Java»

Студент группы ИУ9-21Б Белянин Г. И.

Преподаватель Посевин Д. П.

Москва 2026

1 Цель

Познакомиться с интерфейсами в Java, реализовать веб-страничку для учёта лабораторных работ.

2 Персональный вариант

Задание Реализовать класс, представляющий множество материальных точек, заданных координатами в трёхмерном пространстве и массой, с порядком на основрас- стояния от центра масс до начала координат. Сделать веб-страничку для учёта лабораторных работ и внести туда 3ю и 2ю лабы.

3 Практическая реализация

Сервер Interval

```
import java.util.Objects;

public class Interval {
    private final int a;
    private final int b;

    public Interval(int a, int b) {
        this.a = Math.min(a, b);
        this.b = Math.max(a, b);
    }

    public int getA() {
        return a;
    }

    public int getB() {
        return b;
    }

    public int size() {
        return b - a;
    }

    public boolean contains(double x) {
        return a <= x && x <= b;
    }

    @Override
    public String toString() {
        return "[" + a + "," + b + "]";
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Interval)) return false;
        Interval interval = (Interval) o;
        return a == interval.a && b == interval.b;
    }

    @Override
    public int hashCode() {
        return Objects.hash(a, b);
    }
}
```

```

    }
}

```

Класс IntervalSet

```

import java.util.*;
import java.util.stream.Stream;

public class IntervalSet {
    private final Set<Interval> intervals;

    public IntervalSet() {
        this.intervals = new HashSet<>();
    }

    public void add(Interval interval) {
        intervals.add(interval);
    }

    public void add(int a, int b) {
        intervals.add(new Interval(a, b));
    }

    public Stream<Interval> intervalsContainingRoots(double a, double b,
        double c) {
        List<Double> roots = solveQuadratic(a, b, c);
        if (roots.isEmpty()) {
            return Stream.empty();
        }
        return intervals.stream()
            .filter(interval -> roots.stream().anyMatch(interval::
                contains));
    }

    public Optional<Interval> largestIntervalContainingRoots(double a, double
        b, double c) {
        List<Double> roots = solveQuadratic(a, b, c);
        if (roots.isEmpty()) {
            return Optional.empty();
        }
        return intervals.stream()
            .filter(interval -> roots.stream().anyMatch(interval::
                contains))
            .max(Comparator.comparingInt(Interval::size));
    }

    private List<Double> solveQuadratic(double a, double b, double c) {
        if (a == 0) {

```

```

        return (b != 0) ? Collections.singletonList(-c / b) : Collections
            .emptyList();
    }
    double discriminant = b * b - 4 * a * c;
    if (discriminant < 0) {
        return Collections.emptyList();
    } else if (discriminant == 0) {
        return Collections.singletonList(-b / (2 * a));
    } else {
        double sqrtD = Math.sqrt(discriminant);
        return Arrays.asList(
            (-b - sqrtD) / (2 * a),
            (-b + sqrtD) / (2 * a)
        );
    }
}
}

```

Класс Test

```

import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

public class Test {
    public static void main(String[] args) {
        IntervalSet set = new IntervalSet();

        set.add(2, 2);
        set.add(2, 3);
        set.add(2, 4);
        set.add(1, 4);
        set.add(2, 6);
        set.add(0, 5);
        set.add(2, 8);
        set.add(1, 7);
        set.add(-5, 3);
        set.add(-10, 10);
        set.add(-100, 100);
        set.add(10, 20);

        double A = 1.0;
        double B = -5.0;
        double C = 6.0;

        System.out.println("
            : " + A + "x^2 + " + B + "x + "
            + C + " = 0");
        System.out.println("
            : 2.0 3.0\n");
    }
}

```

```

System.out.println(" ---
                                (10)                                ) ---");

Map<Integer, List<Interval>> grouped = set.intervalsContainingRoots(A
    , B, C)
    .collect( Collectors.groupingBy(interval -> Math.min(interval.
        size(), 9)));

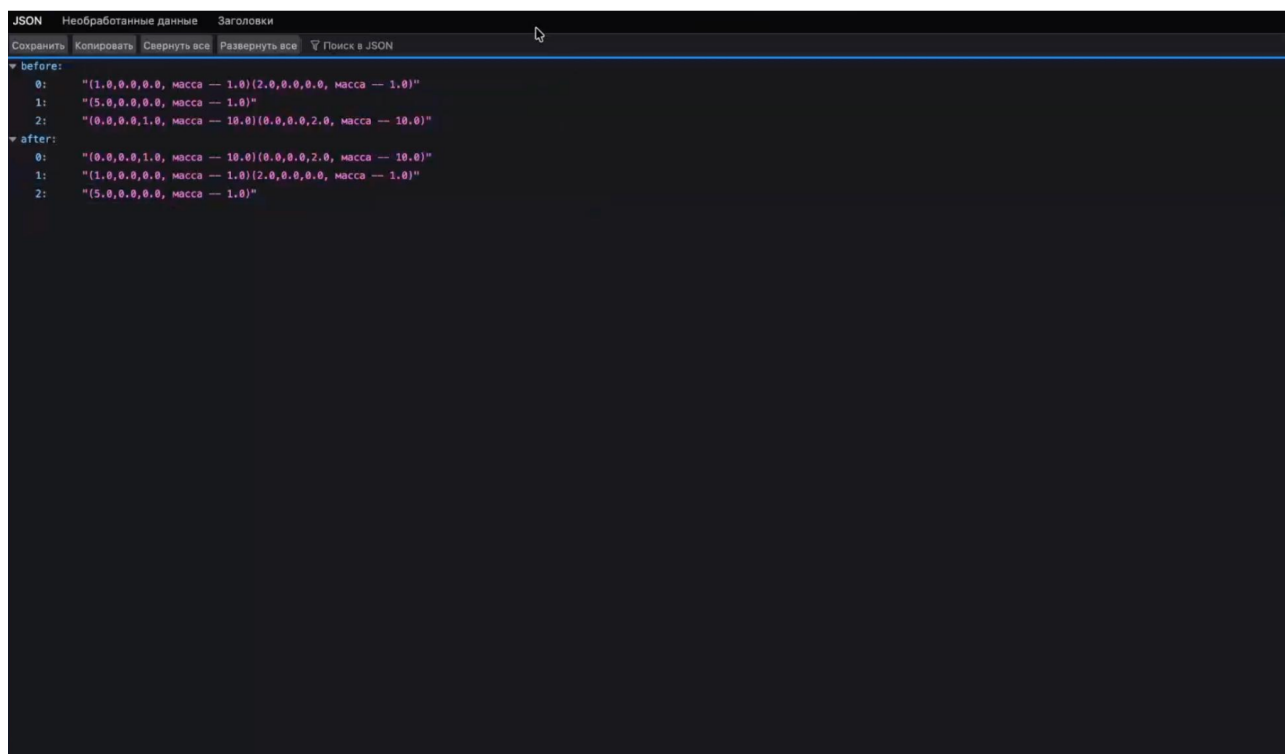
grouped.entrySet().stream()
    .sorted(Map.Entry.comparingByKey())
    .forEach(entry -> {
        String range = (entry.getKey() == 9) ? "[9,    )" : "[" +
            entry.getKey() + ", " + (entry.getKey() + 1) + ")";
        System.out.println("                                " + range +
            ": " + entry.getValue());
    });

System.out.println();
System.out.println(" ---
                                ---");

set.largestIntervalContainingRoots(A, B, C)
    .ifPresentOrElse(
        interval -> System.out.println("                                : " +
            interval + " (                                : " + interval.size() +
                ")"),
        () -> System.out.println("                                ")
    );
}
}

```

4 Результаты



5 Выводы

В ходе лабораторной работы реализована предметная область «Множество интервалов» с применением функциональных возможностей языка Java. Освоены принципы работы с монадами Stream и Optional: реализована фильтрация интервалов, содержащих корни квадратного уравнения, и поиск максимального по длине интервала среди подходящих. Корректность алгоритмов подтверждена тестированием методом группировки результатов по диапазонам длин. Цель работы — приобретение навыков использования монад в Java — достигнута.